

OIG_Python_Practice

April 16, 2026

1 Python for OIG Data Analysis

1.0.1 Practice Notebook — Joseph

Covers: Pandas data manipulation skills for audit and oversight work.

1.1 Setup — Generate Sample Data

Run this cell first. It creates a fake USPS procurement dataset we'll use throughout.

```
[32]: import pandas as pd
import numpy as np

np.random.seed(42)
n = 500

vendors = ["OfficeMax", "Staples", "Dell", "Ford Fleet", "FedEx", "Amazon", "W.
↳W. Grainger"]
districts = ["Northeast", "Southeast", "Midwest", "Southwest", "Pacific"]
categories = ["Office Supplies", "IT Equipment", "Vehicles", "Shipping",
↳"Maintenance"]

df = pd.DataFrame({
    "transaction_id": range(1001, 1001 + n),
    "date": pd.date_range(start="2024-01-01", periods=n, freq="D").to_series().
↳sample(n, replace=True).values,
    "vendor": np.random.choice(vendors, n),
    "district": np.random.choice(districts, n),
    "category": np.random.choice(categories, n),
    "amount": np.round(np.random.exponential(scale=2000, size=n), 2),
    "quantity": np.random.randint(1, 50, n),
    "approved_by": np.random.choice(["Harris", "Patel", "Nguyen", "Brooks",
↳"Okafor"], n)
})

# Inject a few anomalies for later exercises
df.loc[12, "amount"] = 87500.00 # suspiciously large
```

```

df.loc[47, "amount"] = 74200.00
df.loc[99, "amount"] = 9999.99      # just under a common approval threshold
df.loc[100, "amount"] = 9999.99     # duplicate
df.loc[200, "amount"] = 9999.99     # triplicate

# Inject a missing value
df.loc[55, "vendor"] = np.nan

print(f"Dataset ready: {df.shape[0]} rows, {df.shape[1]} columns")
df.head()

```

Dataset ready: 500 rows, 8 columns

```

[32]: transaction_id      date      vendor  district      category \
0          1001 2024-04-12      Staples  Northeast      Maintenance
1          1002 2025-03-11  W.W. Grainger  Southeast  Office Supplies
2          1003 2024-12-14          Dell  Southwest      IT Equipment
3          1004 2024-09-27      Staples  Southeast      Maintenance
4          1005 2024-04-16      Staples  Southeast      Shipping

      amount  quantity  approved_by
0  2597.81         25      Harris
1  2715.08         22      Brooks
2  1108.49         30      Brooks
3   849.10         20      Harris
4   927.37         21      Nguyen

```

1.2 Section 1: Exploring a New Dataset

The first thing you do with any dataset at OIG is **understand what you're looking at**. These are the commands you'll run every single time.

```

[33]: # How big is the dataset?
df.shape

```

[33]: (500, 8)

```

[34]: # What are the column names and data types?
df.dtypes

```

```

[34]: transaction_id      int64
date                    datetime64[us]
vendor                  str
district                str
category                str
amount                  float64
quantity                int32

```

```
approved_by          str
dtype: object
```

```
[35]: # Are there any missing values?
df.isnull().sum()
```

```
[35]: transaction_id    0
      date             0
      vendor           1
      district         0
      category         0
      amount           0
      quantity         0
      approved_by      0
      dtype: int64
```

```
[36]: # Summary statistics - min, max, mean, std for numeric columns
df.describe()
```

```
[36]:
```

	transaction_id	date	amount	quantity
count	500.000000	500	500.000000	500.000000
mean	1250.500000	2024-09-09 02:55:40.800000	2383.339760	25.502000
min	1001.000000	2024-01-02 00:00:00	9.900000	1.000000
25%	1125.750000	2024-05-13 18:00:00	556.555000	14.000000
50%	1250.500000	2024-09-08 00:00:00	1430.065000	25.000000
75%	1375.250000	2025-01-08 06:00:00	2690.457500	37.000000
max	1500.000000	2025-05-14 00:00:00	87500.000000	49.000000
std	144.481833	NaN	5439.635952	13.929899

```
[37]: # How many unique vendors are there?
df["vendor"].nunique()
```

```
[37]: 7
```

```
[38]: # What are the unique values in a column?
df["district"].unique()
```

```
[38]: <StringArray>
      ['Northeast', 'Southeast', 'Southwest', 'Midwest', 'Pacific']
      Length: 5, dtype: str
```

1.2.1 EXERCISE 1

Answer these questions using the tools above:

1. How many transactions are in the dataset?
2. Which column has a missing value, and how many are missing?
3. What is the maximum transaction amount?
4. How many unique approvers (`approved_by`) are there?

Write your code in the cell below:

```
[39]: # Your code here
```

1.3 Section 2: Filtering Rows

Filtering lets you zoom in on exactly the records you care about. Syntax: `df[condition]`

```
[40]: # All transactions over $10,000
df[df["amount"] > 10000]
```

```
[40]:
```

	transaction_id	date	vendor	district	category	amount	\
12	1013	2024-11-26	FedEx	Pacific	Shipping	87500.00	
47	1048	2025-03-21	Staples	Pacific	Maintenance	74200.00	
121	1122	2024-09-08	Amazon	Southwest	IT Equipment	14883.45	
273	1274	2024-04-22	FedEx	Pacific	IT Equipment	10278.99	
372	1373	2024-10-30	FedEx	Southwest	Shipping	10543.86	
384	1385	2025-03-31	Ford Fleet	Midwest	Vehicles	11425.76	

	quantity	approved_by
12	30	Okafor
47	23	Okafor
121	40	Nguyen
273	7	Brooks
372	5	Brooks
384	9	Brooks

```
[41]: # Only transactions from the Northeast district
df[df["district"] == "Northeast"]
```

```
[41]:
```

	transaction_id	date	vendor	district	category	\
0	1001	2024-04-12	Staples	Northeast	Maintenance	
10	1011	2025-04-11	Amazon	Northeast	Vehicles	
14	1015	2024-03-28	W.W. Grainger	Northeast	IT Equipment	
15	1016	2025-01-07	Staples	Northeast	Maintenance	
30	1031	2024-10-03	Ford Fleet	Northeast	Vehicles	
..	...	...	...	...	...	
481	1482	2024-06-09	W.W. Grainger	Northeast	Vehicles	
483	1484	2024-11-18	FedEx	Northeast	IT Equipment	
485	1486	2024-01-18	Staples	Northeast	Vehicles	
492	1493	2024-12-25	Amazon	Northeast	Vehicles	
497	1498	2025-03-06	Ford Fleet	Northeast	Shipping	

	amount	quantity	approved_by
0	2597.81	25	Harris
10	2583.84	35	Harris
14	1095.40	49	Brooks

15	648.22	12	Brooks
30	1355.64	11	Okafor
..	...	...	...
481	2065.11	23	Nguyen
483	85.71	22	Okafor
485	3206.06	5	Patel
492	2137.01	33	Patel
497	930.96	35	Patel

[104 rows x 8 columns]

```
[42]: # Combine conditions with & (AND) and | (OR)
# Note: each condition must be wrapped in parentheses

# IT Equipment purchases over $5,000
df[(df["category"] == "IT Equipment") & (df["amount"] > 5000)]
```

```
[42]:
```

	transaction_id	date	vendor	district	category	amount	\
5	1006	2024-03-12	Staples	Southeast	IT Equipment	8788.13	
39	1040	2025-04-19	Amazon	Southeast	IT Equipment	6974.32	
81	1082	2024-07-24	OfficeMax	Northeast	IT Equipment	8629.50	
84	1085	2024-02-19	Ford Fleet	Midwest	IT Equipment	6065.28	
99	1100	2024-07-20	Staples	Southeast	IT Equipment	9999.99	
121	1122	2024-09-08	Amazon	Southwest	IT Equipment	14883.45	
123	1124	2025-01-14	FedEx	Northeast	IT Equipment	7559.73	
200	1201	2024-08-18	Staples	Northeast	IT Equipment	9999.99	
248	1249	2024-06-13	Dell	Midwest	IT Equipment	9478.77	
273	1274	2024-04-22	FedEx	Pacific	IT Equipment	10278.99	
280	1281	2024-08-07	FedEx	Southwest	IT Equipment	6322.41	
484	1485	2024-05-07	FedEx	Pacific	IT Equipment	6914.82	

	quantity	approved_by
5	44	Brooks
39	18	Nguyen
81	43	Nguyen
84	21	Okafor
99	17	Patel
121	40	Nguyen
123	21	Patel
200	42	Patel
248	25	Harris
273	7	Brooks
280	21	Harris
484	17	Okafor

```
[43]: # isin() - filter for multiple values at once
# Transactions from Dell OR Ford Fleet
```

```
df[df["vendor"].isin(["Dell", "Ford Fleet"])]
```

```
[43]:
```

	transaction_id	date	vendor	district	category	\
2	1003	2024-12-14	Dell	Southwest	IT Equipment	
9	1010	2024-05-01	Dell	Southwest	Office Supplies	
16	1017	2024-04-09	Dell	Pacific	Maintenance	
20	1021	2024-05-29	Ford Fleet	Midwest	Office Supplies	
24	1025	2025-05-06	Ford Fleet	Southeast	Maintenance	
..	...	...	...	...	...	
473	1474	2024-07-06	Ford Fleet	Pacific	Maintenance	
479	1480	2025-02-19	Dell	Southeast	Maintenance	
487	1488	2025-05-04	Dell	Southwest	Office Supplies	
494	1495	2024-10-06	Dell	Southeast	Maintenance	
497	1498	2025-03-06	Ford Fleet	Northeast	Shipping	

	amount	quantity	approved_by
2	1108.49	30	Brooks
9	1154.70	21	Nguyen
16	1794.69	41	Harris
20	1996.96	13	Okafor
24	4794.51	40	Nguyen
..	...	...	...
473	700.38	33	Brooks
479	1909.09	18	Nguyen
487	7821.08	11	Okafor
494	825.79	16	Brooks
497	930.96	35	Patel

```
[131 rows x 8 columns]
```

1.3.1 EXERCISE 2

Write filters to find:

1. All transactions approved by "Harris"
2. All Midwest transactions under \$500
3. All transactions from either the Southwest or Pacific district
4. All Vehicle purchases over \$20,000 (this would be a real audit flag)

Bonus: How many rows does each filter return? (Use `len()` around your filter)

```
[44]: # Your code here
```

1.4 Section 3: Grouping and Aggregating

`groupby()` answers questions like: *which district is spending the most?* It groups rows by a category, then applies a math operation to each group.

```
[45]: # Total spending by district
df.groupby("district")["amount"].sum().sort_values(ascending=False)
```

```
[45]: district
Pacific      355456.07
Southeast    225829.36
Midwest      220208.55
Northeast    211402.51
Southwest    178773.39
Name: amount, dtype: float64
```

```
[46]: # Average transaction size by vendor
df.groupby("vendor")["amount"].mean().sort_values(ascending=False).round(2)
```

```
[46]: vendor
FedEx          3330.74
Staples         3209.80
OfficeMax       2182.76
Amazon          2102.42
Ford Fleet      1899.13
Dell            1858.25
W.W. Grainger   1583.21
Name: amount, dtype: float64
```

```
[47]: # Count of transactions per approver
df.groupby("approved_by")["transaction_id"].count().sort_values(ascending=False)
```

```
[47]: approved_by
Nguyen      114
Okafor      106
Harris      105
Brooks      100
Patel       75
Name: transaction_id, dtype: int64
```

```
[48]: # Multiple stats at once using .agg()
df.groupby("district")["amount"].agg(["sum", "mean", "max", "count"]).round(2)
```

```
[48]:
```

	sum	mean	max	count
district				
Midwest	220208.55	2117.39	11425.76	104
Northeast	211402.51	2032.72	9999.99	104
Pacific	355456.07	3627.10	87500.00	98
Southeast	225829.36	2150.76	9999.99	105
Southwest	178773.39	2008.69	14883.45	89

1.4.1 EXERCISE 3

Use `groupby()` to answer these audit questions:

1. Which **category** has the highest total spending?
2. Which **approver** has approved the highest average transaction amount?
3. How many transactions did each **vendor** have? Who had the most?
4. For the **Pacific** district only, what is the total spending per category? (Hint: filter first, then `groupby`)

```
[49]: # Your code here
```

1.5 Section 4: Adding Computed Columns

You can create new columns by doing math on existing ones. This is useful for deriving metrics like cost-per-unit or flagging thresholds.

```
[50]: # Cost per unit
df["cost_per_unit"] = (df["amount"] / df["quantity"]).round(2)
df[["transaction_id", "vendor", "amount", "quantity", "cost_per_unit"]].head(10)
```

```
[50]:
```

	transaction_id	vendor	amount	quantity	cost_per_unit
0	1001	Staples	2597.81	25	103.91
1	1002	W.W. Grainger	2715.08	22	123.41
2	1003	Dell	1108.49	30	36.95
3	1004	Staples	849.10	20	42.46
4	1005	Staples	927.37	21	44.16
5	1006	Staples	8788.13	44	199.73
6	1007	OfficeMax	81.87	32	2.56
7	1008	OfficeMax	4035.29	46	87.72
8	1009	OfficeMax	1728.70	49	35.28
9	1010	Dell	1154.70	21	54.99

```
[51]: # Boolean flag - True if amount exceeds $10,000
df["high_value"] = df["amount"] > 10000
df["high_value"].value_counts()
```

```
[51]: high_value
False    494
True       6
Name: count, dtype: int64
```

```
[52]: # np.where() - assign a label based on a condition
# Like an IF statement for a whole column
df["risk_level"] = np.where(df["amount"] > 10000, "HIGH", "NORMAL")
df["risk_level"].value_counts()
```

```
[52]: risk_level
      NORMAL    494
      HIGH      6
      Name: count, dtype: int64
```

1.5.1 EXERCISE 4

Add these columns to the DataFrame:

1. `just_under_threshold` — True if the amount is between \$9,000 and \$10,000 (This is a real fraud signal called “threshold avoidance” — splitting or timing purchases to stay just under an approval limit)
2. `approver_flag` — the label "REVIEW" if approved by "Brooks" and amount > \$5,000, otherwise "OK". Use `np.where()`.
3. After creating column 1, how many transactions are flagged as just under threshold?

```
[53]: # Your code here
```

1.6 Section 5: Sorting and Selecting Columns

Clean output matters when you’re presenting findings.

```
[54]: # Sort by amount, largest first
df.sort_values("amount", ascending=False).head(10)
```

```
[54]:
```

	transaction_id	date	vendor	district	category	amount \
12	1013	2024-11-26	FedEx	Pacific	Shipping	87500.00
47	1048	2025-03-21	Staples	Pacific	Maintenance	74200.00
121	1122	2024-09-08	Amazon	Southwest	IT Equipment	14883.45
384	1385	2025-03-31	Ford Fleet	Midwest	Vehicles	11425.76
372	1373	2024-10-30	FedEx	Southwest	Shipping	10543.86
273	1274	2024-04-22	FedEx	Pacific	IT Equipment	10278.99
100	1101	2025-03-21	OfficeMax	Southwest	Vehicles	9999.99
200	1201	2024-08-18	Staples	Northeast	IT Equipment	9999.99
99	1100	2024-07-20	Staples	Southeast	IT Equipment	9999.99
295	1296	2025-02-19	Ford Fleet	Southeast	Vehicles	9781.31

	quantity	approved_by	cost_per_unit	high_value	risk_level
12	30	Okafor	2916.67	True	HIGH
47	23	Okafor	3226.09	True	HIGH
121	40	Nguyen	372.09	True	HIGH
384	9	Brooks	1269.53	True	HIGH
372	5	Brooks	2108.77	True	HIGH
273	7	Brooks	1468.43	True	HIGH
100	14	Harris	714.28	False	NORMAL
200	42	Patel	238.10	False	NORMAL
99	17	Patel	588.23	False	NORMAL

295	4	Okafor	2445.33	False	NORMAL
-----	---	--------	---------	-------	--------

```
[55]: # Select only specific columns
df[["transaction_id", "date", "vendor", "amount", "approved_by"]].head()
```

```
[55]: transaction_id      date      vendor  amount approved_by
0           1001 2024-04-12      Staples  2597.81      Harris
1           1002 2025-03-11  W.W. Grainger  2715.08      Brooks
2           1003 2024-12-14          Dell  1108.49      Brooks
3           1004 2024-09-27      Staples   849.10      Harris
4           1005 2024-04-16      Staples   927.37      Nguyen
```

```
[56]: # Combine: select columns AND sort - a clean audit report view
(
    df[df["amount"] > 10000]
    [["transaction_id", "date", "vendor", "district", "amount", "approved_by"]]
    .sort_values("amount", ascending=False)
    .reset_index(drop=True)
)
```

```
[56]: transaction_id      date      vendor  district  amount approved_by
0           1013 2024-11-26      FedEx    Pacific  87500.00      Okafor
1           1048 2025-03-21      Staples    Pacific  74200.00      Okafor
2           1122 2024-09-08      Amazon  Southwest  14883.45      Nguyen
3           1385 2025-03-31  Ford Fleet    Midwest  11425.76      Brooks
4           1373 2024-10-30      FedEx    Southwest  10543.86      Brooks
5           1274 2024-04-22      FedEx    Pacific  10278.99      Brooks
```

1.6.1 EXERCISE 5 — Putting It All Together

Build a **High-Value Transaction Report**:

1. Filter for transactions over \$8,000
2. Select only: `transaction_id`, `date`, `vendor`, `district`, `amount`, `approved_by`
3. Sort by `amount` descending
4. Reset the index
5. Save the result to a variable called `high_value_report`
6. Print how many transactions are in the report
7. Export it to CSV: `high_value_report.to_csv("high_value_report.csv", index=False)`

```
[57]: # Your code here
```

1.7 Answer Key

Run these cells to check your work — but try the exercises first!

```
[58]: # EXERCISE 1 answers
print("1. Rows:", df.shape[0])
print("2. Missing values:\n", df.isnull().sum()[df.isnull().sum() > 0])
print("3. Max amount:", df["amount"].max())
print("4. Unique approvers:", df["approved_by"].nunique())
```

```
1. Rows: 500
2. Missing values:
   vendor      1
dtype: int64
3. Max amount: 87500.0
4. Unique approvers: 5
```

```
[59]: # EXERCISE 2 answers
print("1. Harris:", len(df[df["approved_by"] == "Harris"]))
print("2. Midwest < $500:", len(df[(df["district"] == "Midwest") &
    ↪(df["amount"] < 500)]))
print("3. SW or Pacific:", len(df[df["district"].isin(["Southwest",
    ↪"Pacific"])]))
print("4. Vehicle > $20k:", len(df[(df["category"] == "Vehicles") &
    ↪(df["amount"] > 20000)]))
```

```
1. Harris: 105
2. Midwest < $500: 22
3. SW or Pacific: 187
4. Vehicle > $20k: 0
```

```
[60]: # EXERCISE 3 answers
print("1. Highest spending category:")
print(df.groupby("category")["amount"].sum().sort_values(ascending=False).
    ↪head(1))
print("\n2. Highest avg approver:")
print(df.groupby("approved_by")["amount"].mean().sort_values(ascending=False).
    ↪head(1).round(2))
print("\n3. Transactions per vendor:")
print(df.groupby("vendor")["transaction_id"].count().
    ↪sort_values(ascending=False))
print("\n4. Pacific spending by category:")
print(df[df["district"] == "Pacific"].groupby("category")["amount"].sum().
    ↪sort_values(ascending=False).round(2))
```

```
1. Highest spending category:
category
Shipping      259156.94
Name: amount, dtype: float64
```

```
2. Highest avg approver:
approved_by
```

```
Okafor      3456.6
Name: amount, dtype: float64
```

3. Transactions per vendor:

```
vendor
FedEx      92
OfficeMax  77
Staples    76
Ford Fleet 74
W.W. Grainger 65
Amazon     58
Dell       57
Name: transaction_id, dtype: int64
```

4. Pacific spending by category:

```
category
Shipping      117572.65
Maintenance    92613.50
Office Supplies 59012.63
Vehicles       46913.51
IT Equipment   39343.78
Name: amount, dtype: float64
```

```
[61]: # EXERCISE 4 answers
df["just_under_threshold"] = (df["amount"] >= 9000) & (df["amount"] <= 10000)
df["approver_flag"] = np.where((df["approved_by"] == "Brooks") & (df["amount"] > 5000), "REVIEW", "OK")
print("Just under threshold:", df["just_under_threshold"].sum())
df[df["just_under_threshold"]][["transaction_id", "amount", "approved_by"]]
```

Just under threshold: 6

```
[61]:      transaction_id  amount approved_by
99              1100  9999.99         Patel
100             1101  9999.99         Harris
200             1201  9999.99         Patel
248             1249  9478.77         Harris
295             1296  9781.31         Okafor
321             1322  9696.53         Okafor
```

```
[62]: # EXERCISE 5 answer
high_value_report = (
    df[df["amount"] > 8000]
    [["transaction_id", "date", "vendor", "district", "amount", "approved_by"]]
    .sort_values("amount", ascending=False)
    .reset_index(drop=True)
)
print(f"High-value transactions: {len(high_value_report)}")
```

```
high_value_report.to_csv("high_value_report.csv", index=False)
print("Exported to high_value_report.csv")
high_value_report.head()
```

High-value transactions: 16

Exported to high_value_report.csv

```
[62]:
```

	transaction_id	date	vendor	district	amount	approved_by
0	1013	2024-11-26	FedEx	Pacific	87500.00	Okafor
1	1048	2025-03-21	Staples	Pacific	74200.00	Okafor
2	1122	2024-09-08	Amazon	Southwest	14883.45	Nguyen
3	1385	2025-03-31	Ford Fleet	Midwest	11425.76	Brooks
4	1373	2024-10-30	FedEx	Southwest	10543.86	Brooks